

How a command becomes durable

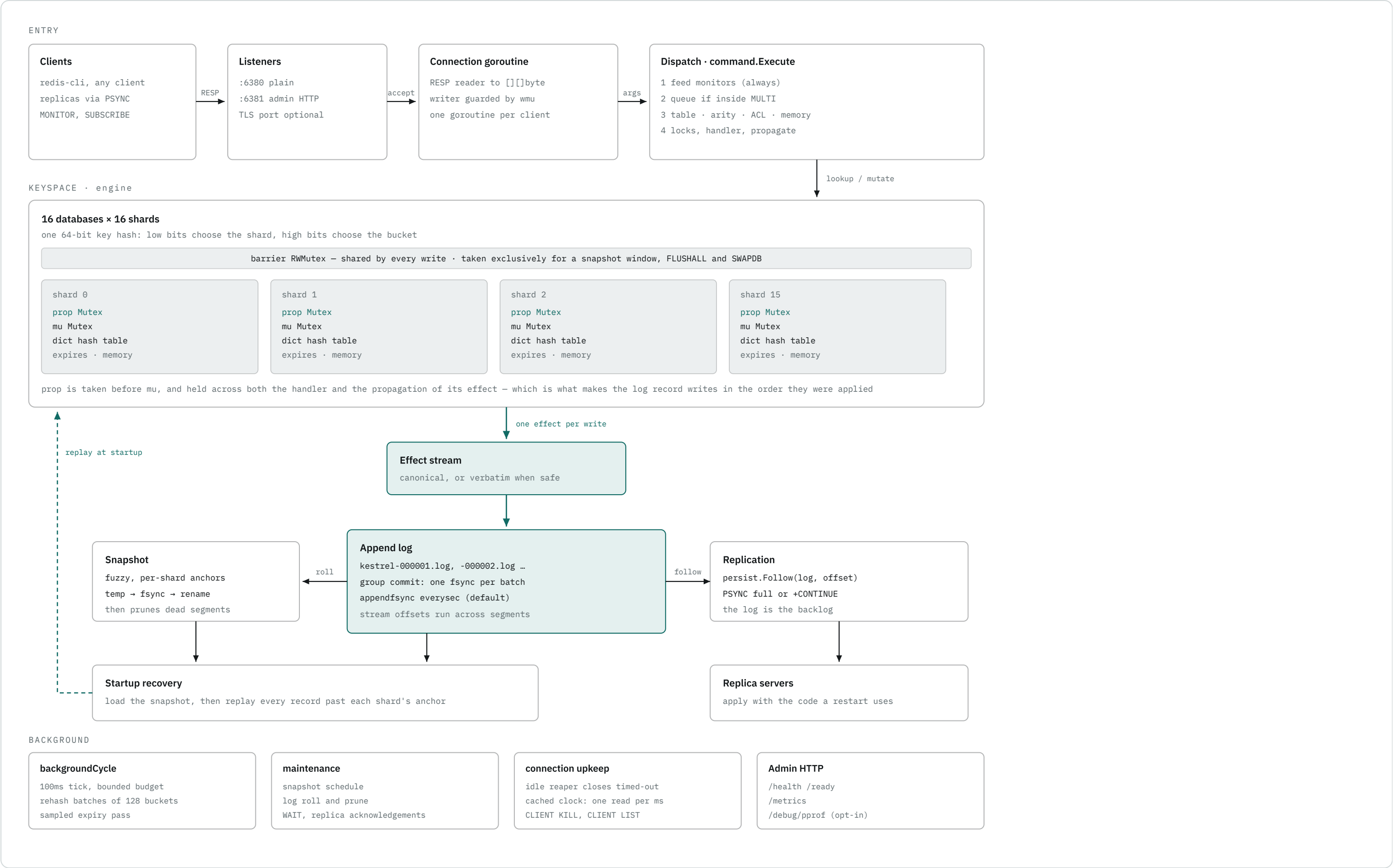
One diagram of the whole machine, then the four mechanisms it has to compress: the two locks a write holds, the single hash behind sharding, and what a restart reads back.

LISTENER :6380	ADMIN HTTP :6381	KEYSPACE 16 × 16	COMMANDS 200	DURABILITY everysec	PACKAGES 8
-------------------	---------------------	---------------------	-----------------	------------------------	---------------

THE WHOLE PATH

Client to disk, and back out to replicas

Every write takes the same route. It arrives on a connection that owns a goroutine, passes four gates in the dispatcher, mutates one shard under two locks, and leaves behind a single replay-safe *effect*. That effect is the only thing the rest of the system consumes: the append log writes it, the snapshotter anchors against it, and replicas are fed by following the same log. There is no second copy and no separate replication buffer.



Two locks, and why one of them is held longer

A shard has two mutexes. `mu` covers the mutation. `prop` covers the mutation *and* the moment its effect reaches the log. Without the second one, two writers can mutate in one order and append in the other — and nothing notices until a restart rebuilds a dataset that was never running.

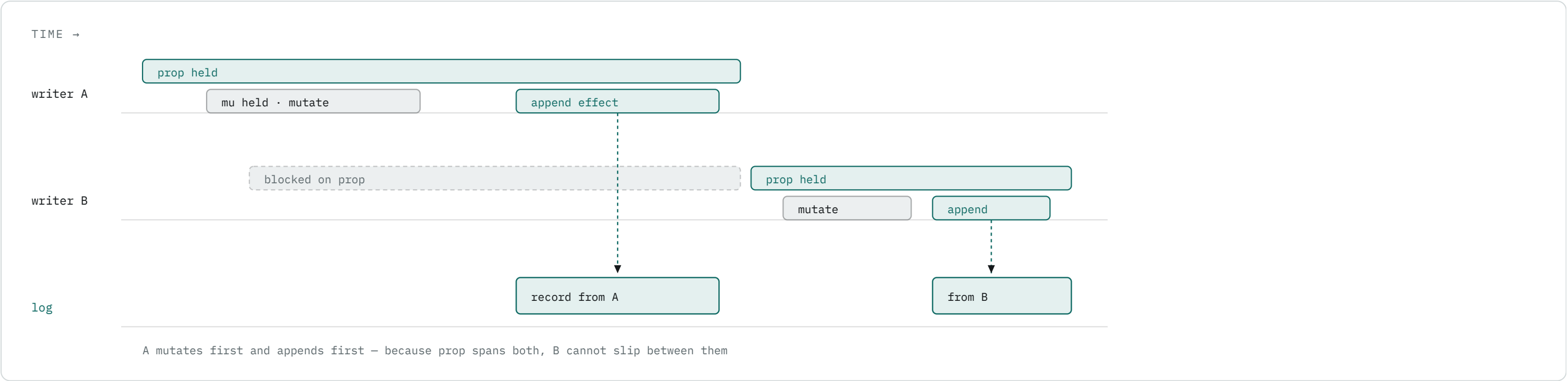


Figure 2. Writer B is blocked on `prop` for the whole of A's span, not just A's mutation. Reads take neither lock, which is why `GET` stays on the zero-allocation path — and why a read that finds an expired key hides it rather than deleting it: a read has no ordered position in this picture, so it must not write to the stream.

One hash, used twice

A key is hashed once. The low bits pick the shard; the high bits pick the bucket inside that shard's table. Taking them from opposite ends matters: every key in a shard shares its low bits, so a table indexed by those would put the whole shard in one bucket.

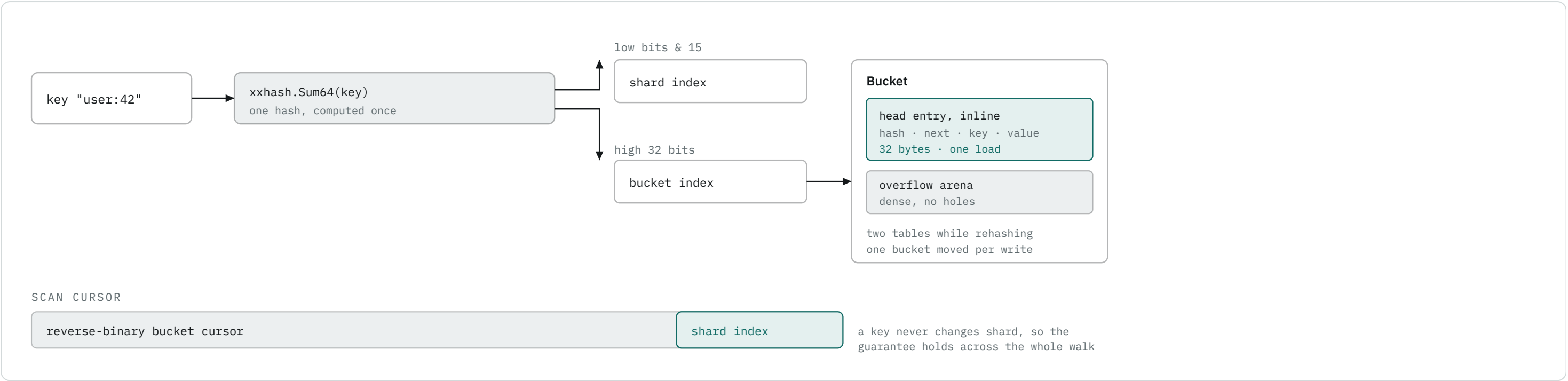


Figure 3. The head of every chain lives in the bucket array itself, so the common lookup is one dependent load rather than two — measured at 669 ns against Go's own map at 725 ns for a million keys in random order. The cursor packs the shard into its low bits, which works only because a key's shard is fixed for the life of the process.

MECHANISM THREE

What a restart reads back

The snapshot is taken shard by shard while the server keeps serving, so different shards are captured at different points in the stream. Each one records the offset it corresponds to — its *anchor* — and recovery replays a record only if it falls past the anchor of the shard it belongs to. Replaying everything would double-apply an `INCR` ; replaying nothing would lose it.

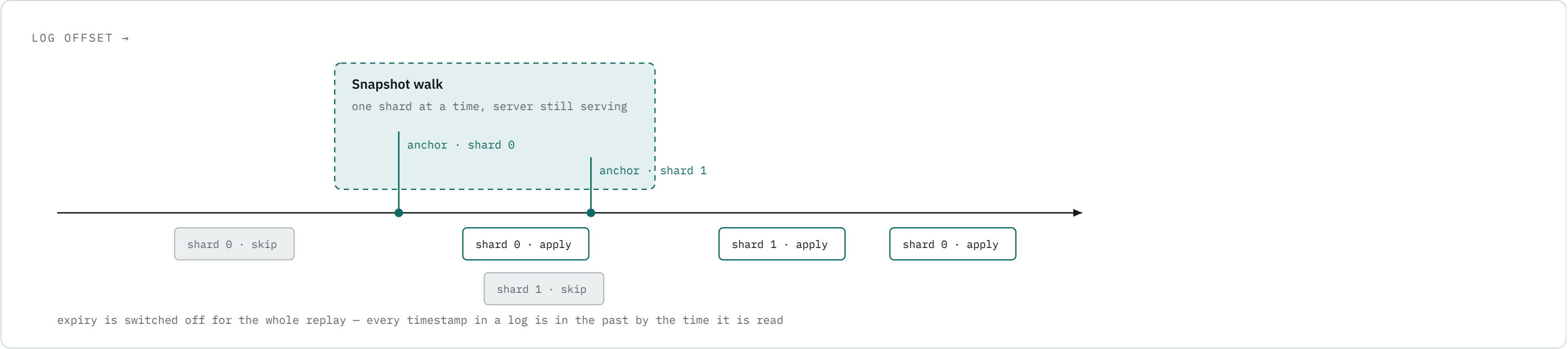


Figure 4. A record is replayed only when it sits past the anchor of *its own* shard, which is why the two shards here accept and reject different parts of the same stream. The snapshot writer takes the barrier exclusively, so a cross-shard write cannot straddle the window.

WHAT THE SHAPE BUYS

Guarantees that fall out of the layout

write ordering	The log records writes in the order they were applied, per key, because <code>prop</code> spans the handler and the append.
durability	An append returns only once the batch carrying its record is written and forced. A failed batch is reported as failed to every caller whose record it carried.
snapshot ≠ pause	Shards are walked one at a time, so the server keeps serving. The cost is per-shard anchors, which recovery then has to respect.
scan	A key present for the whole iteration comes back at least once, however many times the tables resized underneath the cursor. <code>COUNT</code> bounds the reply from above.
reads are silent	A read takes no ordering lock, so it never appends. An expired key is hidden on read and deleted by the background cycle, which does hold the lock.
replication	Replicas are fed by following the append log, so the backlog is the log. A replica that reconnects within retention resumes with <code>+CONTINUE</code> .

BOUNDARIES

What this diagram does not contain

Honest gaps, so nothing in the picture is read as a promise it does not make.

ABSENT	STATUS
Streams, scripting, cluster mode, geo, HyperLogLog	Out of scope by decision. Each returns an error naming the decision rather than “unknown command”.
Bit operations – SETBIT, GETBIT, BITCOUNT, BITPOS, BITOP	Not implemented, and not yet listed in the deviations document. A real gap.
ROLE, MOVE, COMMAND LIST	Not implemented. ROLE is the notable one: replication works, but clients cannot ask about it.
RDB and AOF file compatibility	Kestrel’s snapshot and log are its own formats. Behavioural compatibility is the target, not file compatibility.

PACKAGE GRAPH

Who is allowed to import whom

engine

The keyspace, the value types, expiry, eviction, snapshots. Imports nothing above it — enforced by a test, so the engine stays usable as a library.

server

Connections, both sides of replication, the snapshot schedule, the admin endpoints. The only package that knows about sockets.

persist

Segmented append log, group commit, snapshot files, recovery. Standard library only, and it encodes RESP2 itself so the log’s dialect cannot follow a client’s `HELLO 3`.

resp

The wire protocol, RESP2 and RESP3, as a typed reply tree. Fuzzed in both directions.

command

The table of 200 commands, dispatch, transactions, Pub/Sub, blocking, ACL. Every write must declare its effect kind and its locality, or registration panics.

difftest

Runs the same randomized command stream against Kestrel and a real `redis-server` and compares the replies. Off unless `DIFFTEST=1`.